Java OOP Study Guide

ICD122 / ICD125 - Arrays, Encapsulation, Inheritance, Polymorphism, JFrame

How to use this guide: read a section, then open `OOPStudyGuide.java`, run it, and match each line of output to the code. Every concept is explained in three steps: **what happens**, **why**, and **how to type it**.

The 4 questions to ask before typing any class

Question	Answer becomes
1. What DATA must I store?	private fields
2. What can be DONE with the data?	public methods (getters, setters, actions)
3. Do two classes share data/behaviour?	inheritance: class B extends A
4. Same action, different behaviour per type?	overriding + a parent-typed reference (polymorphism)

Contents

1. Arrays and the for loop: length vs length-1
2. Encapsulation, getters and setters
3. Inheritance
4. Polymorphism
5. JFrame: all three together
6. Abstract classes
7. Interfaces
8. ArrayList
9. Practice exercises with answers
10. Cheat sheet and common mistakes

1. Arrays and the for loop: length vs length-1

In plain words: An array is a row of numbered boxes starting at 0. The loop must stop before it falls off the end.

What happens in memory

```
int[] ar = new int[5];      // 5 boxes, all start as 0
ar[0]=10; ar[1]=20; ar[2]=30; ar[3]=40; ar[4]=50;

index:   0    1    2    3    4
value:  10   20   30   40   50

ar.length     = 5   (how many items)
ar.length - 1 = 4   (index of the LAST item)
```

Golden rule: indexes run from **0** to **length-1**. There is no ar[5].

Three loops, three results

```
for (int i = 0; i < ar.length;     i++)  // A
for (int i = 0; i < ar.length - 1; i++)  // B
for (int i = 0; i <= ar.length;    i++)  // C
```

Loop	i takes values	Condition stops when	Result
A: i < length	0,1,2,3,4	i=5, 5<5 false	Visits ALL 5 items (normal loop)
B: i < length-1	0,1,2,3	i=4, 4<4 false	Misses the LAST item (50 never printed)
C: i <= length	0,1,2,3,4,5	i=6	CRASH: ArrayIndexOutOfBoundsException at ar[5]

So when do you really use length-1?

Use `length-1` when the loop body touches the **next** element, `ar[j+1]`. If j reached the last index, j+1 would be out of range, so j must stop one earlier. This is the bubble sort in Q71:

```
int[] marks = {56, 76, 36, 96};
for (int i = 0; i < marks.length - 1; i++) {        // number of passes
    for (int j = 0; j < marks.length - 1; j++) {    // j+1 must exist
        if (marks[j] < marks[j+1]) {                // compare neighbours
            int temp = marks[j];                    // swap with temp
            marks[j] = marks[j+1];
            marks[j+1] = temp;
        }
    }
}
// result: 96 76 56 36
```

Trace of one pass (j from 0 to 2)

j	compare	swap?	array after
0	56 < 76	yes	76 56 36 96
1	56 < 36	no	76 56 36 96
2	36 < 96	yes	76 56 96 36

Using the last index directly (growing an array)

Arrays cannot grow, so the lecture note builds a bigger copy (extendsStudentArray). The new item always goes to `length-1` of the NEW array:

```
int[] bigger = new int[data.length + 1];            // one more box
for (int i = 0; i < data.length; i++)               // copy old items (A loop)
    bigger[i] = data[i];
bigger[bigger.length - 1] = 4;                      // last box = new item
data = bigger;
```

How to decide while typing: (1) Do I need every item? use `i < length`. (2) Do I use [i+1]? use `i < length-1`. (3) Do I want the last item? use `[length-1]`. (4) Never write <= with length.

2. Encapsulation, getters and setters

In plain words: Keep the data in a locked box (private). Give people a controlled door (getter to look, setter to change).

Encapsulation means: hide the data with `private`, and allow access only through public methods. A **setter** writes a value (and can validate it). A **getter** reads a value.

```
class Box {
    private int length;                        // hidden from outside

    public void setLength(int length) {        // SETTER
        if (length > 0) {                      // validation
            this.length = length;
        }
    }
    public int getLength() {                   // GETTER
        return length;
    }
}
```

Why this.length = length ?

Inside `setLength(int length)` the plain name *length* means the **parameter**. `this.length` means the **field of the current object**. Writing `length = length` copies the parameter to itself and the field never changes (Q13 in your note).

What happens step by step

```
Box b1 = new Box(12,5,3);   // 1. object created, constructor runs
b1.length = 100;            // 2. COMPILE ERROR: length is private
b1.setLength(120);          // 3. allowed: setter checks 120 > 0, stores it
b1.setLength(-5);           // 4. setter rejects it, field stays 120
```

How to type it: (1) write the fields as private. (2) For each field write setX and getX. (3) Put the checks inside the setter. (4) Make the constructor call the setters so validation is reused. (5) Override toString() so System.out.println(obj) is readable.

3. Inheritance

In plain words: A child class copies everything from its parent and then adds or changes what it needs. Write shared code once.

`class Car extends Vehicle` means **Car IS-A Vehicle**. Car gets Vehicle's fields and methods for free, can add its own, and can override methods.

```
class Vehicle {
    protected String id;
    Vehicle(String id) { this.id = id; System.out.println("Vehicle()"); }
    public void park() { System.out.println("Vehicle parking"); }
}
class Car extends Vehicle {
    private int doors;
    Car(String id, int doors) {
        super(id);                 // 1st line: builds the Vehicle part
        this.doors = doors;
        System.out.println("Car()");
    }
    @Override
    public void park() { System.out.println("Car parking"); }
}
```

Order of events for new Car("CAR-1", 4)

Step	What runs
1	Car constructor starts, immediately jumps to super(id)
2	Vehicle constructor runs: id is set, prints Vehicle()
3	Back in Car: doors is set, prints Car()

Rules to remember

Rule	Example / note
Parent constructor always runs first	If you do not write super(...), the compiler inserts super();
No no-arg constructor in parent	Child MUST call super(args) explicitly, otherwise compile error (Q176-Q178)
Private members are not directly accessible	Child uses getters, or the parent uses protected
Overriding	Same name, same parameters, same return type. Use @Override
Overloading is different	Same name, different parameters (Q187). Not overriding
Static members	Belong to the class, not overridden

How to type it: (1) find what two classes share. (2) Put the shared fields and methods in the parent. (3) Write `class Child extends Parent`. (4) In each child constructor call super(...) first. (5) Override only the methods whose behaviour differs.

4. Polymorphism

In plain words: You hold different objects with the same type of remote control. Press the same button, each object reacts in its own way.

Polymorphism means **many forms**: a parent-typed variable can hold any child object, and Java runs the **child's** version of an overridden method. Java chooses the method at **runtime** (dynamic method dispatch).

```
Vehicle v1 = new Car("CAR-1", 4);
Vehicle v2 = new Bus("BUS-1");
v1.park();    // Car parking   (Car's version)
v2.park();    // Bus parking   (Bus's version)
```

The real power: one loop, many behaviours

```
class CarPark {
    private Vehicle[] vehicles = new Vehicle[0];
    public void addVehicle(Vehicle v) { /* grow array, put at length-1 */ }
    public void parkAll() {
        for (Vehicle v : vehicles) {
            v.park();              // same call, different result
        }
    }
}
// Tomorrow you add class Van extends Vehicle.
// CarPark does NOT change. That is extensibility.
```

Legal vs illegal

Code	Legal?	Why
Vehicle v = new Car(...)	Yes	Car IS-A Vehicle (child into parent)
Car c = new Vehicle(...)	No	A Vehicle is not necessarily a Car
v.doors (v is Vehicle type)	No	The reference type decides what you can ACCESS
Car c = (Car) v;	Yes	Cast is fine when the real object is a Car

Two rules in one line: the **reference type** decides what you are allowed to call; the **object type** decides which version actually runs.

Same idea in the water-level example (Q200)

`WaterLevelObserver` is the parent. Alarm, Display, SMSSender extend it and override `update(int)`. ControlRoom stores them all in one `WaterLevelObserver[]` and calls `ob.update(level)`. Adding a Splitter needs zero changes in ControlRoom.

5. JFrame: all three together

In plain words: A window is just a class. It extends JFrame, hides its parts, and reacts to clicks through a listener.

A form is just a class. Treat it like any other class and the three ideas appear naturally.

Idea	Where it shows in a form
Inheritance	`class AddStudentForm extends JFrame`: setTitle, setSize, add... are inherited, so you call them without a variable name
Encapsulation	JTextField, JButton, JLabel are `private`. Other classes use `getStudentFromForm()` and `setStudentToForm(s)`
Polymorphism	`addActionListener(ActionListener l)` accepts any object of a class implementing ActionListener. When clicked, Java calls YOUR actionPerformed

Skeleton to type

```
class AddStudentForm extends JFrame {
    private JTextField txtId, txtName, txtPrf, txtDbms;   // private
    private JButton btnAdd;

    AddStudentForm() {
        setTitle("Add Student Form");        // inherited
        setSize(420, 300);
        setDefaultCloseOperation(DISPOSE_ON_CLOSE);
        setLocationRelativeTo(null);

        txtId = new JTextField();            // create components
        ...
        JPanel form = new JPanel(new GridLayout(4,2,4,4));
        form.add(new JLabel("Student ID")); form.add(txtId);
        ...
        add("Center", form);                 // BorderLayout region

        btnAdd = new JButton("Add Student");
        btnAdd.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent evt) {
                Student s = getStudentFromForm();
                ...
            }
        });
    }

    public Student getStudentFromForm() {            // getter style
        return new Student(txtId.getText(), txtName.getText(),
            Integer.parseInt(txtPrf.getText()),      // String -> int
            Integer.parseInt(txtDbms.getText()));
    }
    public void setStudentToForm(Student s) {        // setter style
        txtId.setText(s.getId());
        ...
    }
}
```

What happens when the user clicks the button

Step	Event
1	User clicks Add Student

2	Swing calls actionPerformed() of the listener object you registered
3	getStudentFromForm() reads the text fields and parses numbers
4	A Student object is created (a bad number throws NumberFormatException, so wrap in try/catch)
5	Result is shown or stored in the collection

Layout quick guide

Layout	Behaviour
BorderLayout (JFrame default)	5 regions: North, South, East, West, Center. One component per region
FlowLayout (JPanel default)	Left to right, wraps like text
GridLayout(rows, cols)	Equal sized cells, filled row by row
Panel inside panel	Put a GridLayout JPanel into the Center of a BorderLayout

6. Abstract classes

In plain words: a parent that is only an idea. It says WHAT every child must do, and each child decides HOW.

```
abstract class Shape {
    private String name;
    Shape(String name) { this.name = name; }
    public abstract double area();          // no body: children MUST write it
    public void printInfo() {               // normal method, shared
        System.out.println(name + " area = " + area());
    }
}
class Circle extends Shape {
    private double radius;
    Circle(double r) { super("Circle"); radius = r; }
    public double area() { return Math.PI * radius * radius; }
}
class Rectangle extends Shape {
    private double w, h;
    Rectangle(double w, double h) { super("Rectangle"); this.w = w; this.h = h; }
    public double area() { return w * h; }
}
```

Step by step

Step	What happens
1	Shape s = new Shape(...) is ILLEGAL, an abstract class cannot be created
2	Shape[] shapes = { new Circle(2), new Rectangle(3,4) } is legal (polymorphism)
3	sh.printInfo() runs the shared method from Shape
4	Inside it, area() is called: Java runs the Circle or Rectangle version

How to type it: (1) write the word abstract before class. (2) For behaviour that differs, write the method header with abstract and end with ; (3) For shared behaviour write a normal method. (4) Each child writes extends and overrides every abstract method, or the child must also be abstract.

7. Interfaces

In plain words: a promise list. A class that implements an interface promises to have those methods. Unrelated classes can make the same promise.

```
interface Printable { void print(); }
interface Savable   { void save();  }

class Invoice implements Printable, Savable {    // many interfaces allowed
    public void print() { System.out.println("Printing invoice"); }
    public void save()  { System.out.println("Saving invoice");  }
}
class Report implements Printable {
    public void print() { System.out.println("Printing report"); }
}

Printable[] items = { new Invoice(), new Report() };
for (Printable p : items) p.print();             // polymorphism
```

	extends (class)	implements (interface)
How many	only ONE parent	as many as you like
Contains	fields, finished methods	method headers (a contract)
Meaning	IS-A (Car is a Vehicle)	CAN-DO (Invoice can be printed)
Seen earlier	JFrame, Vehicle	ActionListener on button clicks

Abstract class or interface? Related classes sharing code and data: abstract class. Unrelated classes sharing an ability: interface.

8. ArrayList

In plain words: an array that grows by itself. It does what extendsArray(), add(index) and remove(index) did by hand in Q202-Q218.

By hand (lecture note)	ArrayList
extendsArray(); a[a.length-1] = x;	list.add(x);
add(index, x) with shifting loops	list.add(index, x);
remove(index) with shifting loops	list.remove(index);
a.length	list.size()
a[i]	list.get(i)
a.length == 0	list.isEmpty()

```
import java.util.ArrayList;

ArrayList<Student> list = new ArrayList<Student>();   // only Students allowed
list.add(new Student("S0001","Nimal",56,87));         // at the end
list.add(new Student("S0002","Amal",76,97));
list.add(1, new Student("S0003","Bimal",36,47));      // insert at index 1

for (int i = 0; i < list.size(); i++) {               // last index = size()-1
    System.out.println(list.get(i).getName());        // Nimal Bimal Amal
}
for (Student s : list) {                              // for-each, no index
    System.out.println(s);
}
list.remove(1);                                       // Bimal removed
```

Why learn the manual way first? It shows what ArrayList does inside: make a bigger array, copy, insert. Exams ask for the manual version. The angle brackets <Student> mean the list holds only Student objects, so no casting is needed (unlike the Object[] Collection in Q218).

9. Practice exercises with answers

Method: cover the answer, write the code yourself, predict the output, then compare. All answers are in `OOPStudyGuide.java` (Part 9) and run.

Exercise 1 (arrays)

Given `int[] a = {5, 8, 2, 9};` print the largest value and the sum.

Thinking: assume the first item is the biggest, then check the rest. Start the loop at 1 because a[0] is already used.

```
int max = a[0];
int sum = a[0];
for (int i = 1; i < a.length; i++) {
    if (a[i] > max) max = a[i];
    sum += a[i];
}
// max = 9, sum = 24
```

i	a[i]	max after	sum after
start	-	5	5
1	8	8	13
2	2	8	15
3	9	9	24

Exercise 2 (encapsulation)

Write ScoreCard with a private marks field. setMarks accepts only 0 to 100. getGrade returns A for 75 or more, B for 50 or more, otherwise F.

```
class ScoreCard {
    private int marks;
    public void setMarks(int marks) {
        if (marks >= 0 && marks <= 100) this.marks = marks;
    }
    public int getMarks() { return marks; }
    public String getGrade() {
        if (marks >= 75) return "A";
        if (marks >= 50) return "B";
        return "F";
    }
}
// setMarks(80) -> grade A.   setMarks(150) is rejected, marks stays 80.
```

Exercise 3 (inheritance and polymorphism)

Employee has name and baseSalary. Manager adds a bonus. Developer is paid 500 per overtime hour. Put both in an Employee[] and print every salary with one loop.

```
class Employee {
    protected String name;  protected double baseSalary;
    Employee(String name, double baseSalary) { this.name = name; this.baseSalary = baseSalary; }
    public double getSalary() { return baseSalary; }
    public String getName()   { return name; }
}
class Manager extends Employee {
    private double bonus;
```

```
    Manager(String n, double b, double bonus) { super(n, b); this.bonus = bonus; }
    @Override public double getSalary() { return baseSalary + bonus; }
}
class Developer extends Employee {
    private int overtime;
    Developer(String n, double b, int overtime) { super(n, b); this.overtime = overtime; }
    @Override public double getSalary() { return baseSalary + overtime * 500; }
}
Employee[] staff = { new Manager("Kamal",100000,25000), new Developer("Nimali",90000,10) };
for (Employee e : staff) System.out.println(e.getName() + " " + e.getSalary());
// Kamal 125000.0     Nimali 95000.0
```

More to try on your own

a) Add an abstract method to Employee and make both children implement it.

b) Create an interface Payable with pay() and implement it in Invoice and Employee.

c) Store Students in an ArrayList, then print only those with PRF marks above 60.

d) Rewrite exercise 1 using i < a.length-1 comparing a[i] and a[i+1] to find the largest neighbour pair sum.

10. Cheat sheet and common mistakes

Mistake	Fix
for (i = 0; i <= ar.length; ...)	Use i < ar.length
Using ar[ar.length]	Last item is ar[ar.length-1]
length = length; inside a setter	this.length = length;
Child constructor fails to compile	Parent has no no-arg constructor: add super(args) as the first line
Calling child-only method on parent reference	Cast: ((Car) v).method()
Changing a method signature and expecting overriding	Overriding needs the exact same parameters. Add @Override to catch this
Reading text field as int	Integer.parseInt(txt.getText()) and catch NumberFormatException
Public fields in a form	Make them private and add getters/setters
new on an abstract class or interface	Not allowed. Create a child object instead
Forgot to override an abstract method	Child must implement it or be abstract itself
list.length or list[i] on ArrayList	Use list.size() and list.get(i)

Practice order (do these yourself without looking)

1. Print an array with all three loop types and predict the output first.
2. Write a Student class with private fields, constructor, getters, setters with validation (marks 0 to 100), toString.
3. Create Shape with area(), then Circle and Rectangle that override it. Store them in a Shape[] and print every area.
4. Build an AddStudentForm that extends JFrame and shows a Student object's total when the button is clicked.

Tip: before running, write the expected output on paper. When your prediction and the real output differ, that gap is exactly what you still need to learn.